SRM INSTITUTE OF SCIENCE & TECHNOLOGY, NCR CAMPUS APRIL 2026

[ShipSmart]

A PROJECT REPORT

Submitted by

Shikha Sharma [RA2411030030060]

Aditya Mukherji [RA2411030030052]

Under the guidance of

Dr. Anirudh Gaikwad

(Associate Professor, Department of Computer Science & Engineering)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

SRM INSTITUTE OF SCIENCE & TECHNOLOGY

(Under Section 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this report titled "**ShipSmart**" is the Bonafide work of " **Shikha Sharma [RA2411030030060]"** who carried out this work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other report on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. Neelam Singh
HEAD OF THE DEPARTMENT
Dept. of CSE

SIGNATURE

Dr. Anirudh Gaikwad
Assistant Professor
Dept. of CSE

BONAFIDE CERTIFICATE

Certified that this report titled "**ShipSmart**" is the Bonafide work of " **Aditya Mukherji [RA2411030030052]**", who carried out this work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other report on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. Neelam Singh
HEAD OF THE DEPARTMENT
Dept. of CSE

SIGNATURE

Dr. Anirudh Gaikwad
Assistant Professor
Dept. of CSE

Index

Chapters.	Topic	Page No.
1	Introduction	4
2	Project Description	5-7
3	Programming	8-17
4	Results	18-21
5	Conclusion	22
6	References	23

Chapter 1 : INTRODUCTION

The rapid expansion of e-commerce and digital services has significantly increased the demand for efficient logistics and management systems. Modern businesses require platforms that can seamlessly integrate multiple stakeholders, including sellers, buyers, and administrators, into a single, unified system. This project, titled Logistics and E-Commerce Management System, is designed to address these needs by providing a structured and scalable web-based solution.

The primary objective of this system is to simplify and automate the process of product management, order placement, and delivery tracking. In traditional systems, these operations are often handled manually or through disconnected platforms, leading to inefficiencies, data inconsistencies, and delays. This application eliminates such issues by using a centralized database and well-defined workflows.

The system is developed using a full-stack approach, where the frontend is built using React (with Vite for fast development) and styled using Tailwind CSS for a modern and responsive user interface. The backend is implemented using Node.js and Express, which handle API requests and business logic, while MongoDB serves as the database for storing and managing application data. Communication between frontend and backend is achieved through RESTful APIs using Axios.

The application supports three distinct user roles: Admin, Seller, and Buyer. Each role has specific functionalities and access permissions. Sellers can manage products by adding, viewing, and deleting items. Buyers can browse products and place orders. Administrators have full control over the system, including monitoring all orders, approving them, assigning delivery agents, and marking deliveries as complete.

A key feature of this system is the structured order lifecycle, which ensures that every order progresses through defined stages such as pending approval, approved, out for delivery, and delivered. This improves transparency and accountability within the system.

In addition to implementing core functionalities, the project also demonstrates important database management concepts such as schema design, relationships, and CRUD operations. It highlights real-world challenges like API integration, data consistency, and error handling.

Overall, this project serves as a practical example of how modern web technologies and database systems can be combined to build an efficient, scalable, and user-friendly logistics management platform.

Chapter 2 :PROJECT DESCRIPTION

The Logistics and E-Commerce Management System is designed to perform essential operations involved in managing products, orders, and deliveries in a simplified and structured manner. The application operates through a web-based interface and uses a centralized MongoDB database for storing and managing data efficiently.

System Overview
The system consists of three main roles – Admin, Seller, and Buyer.
A Seller can manage products by adding, viewing, and deleting items available for sale.
A Buyer can browse products and place orders easily through the interface.
An Admin oversees the entire system by managing orders, approving them, assigning delivery agents, and ensuring successful delivery.
The system ensures a smooth workflow by maintaining a structured order lifecycle from creation to final delivery.

Modules:
1. Login Module
- Allows users to select their role (Admin / Seller / Buyer)
- Redirects users to their respective dashboards

2. Seller Module
- Add new products (name, price, category, stock)
- View product list
- Delete products
- View dashboard statistics (total products, inventory value)

3. Buyer Module
- View available products
- Create orders
- Track order status

4. Admin Module
- View all orders
- Approve orders
- Assign delivery agents
- Mark orders as delivered
- Manage delivery agents

5. Order Management Module
- Handles complete order lifecycle:
- Pending Approval

- Approved
- Out for Delivery
- Delivered

6. Data Management Module
- Uses MongoDB for storing:
  - -Users
  - -Products
  - -Orders
  - -Delivery Agents

Functional Requirements:
- Role-based access to system features
- Add, view, and delete products
- Create and manage orders
- Track order lifecycle
- Assign delivery agents to orders
- Maintain product and order records in database

Non-Functional Requirements:
- User-friendly and responsive interface
- Efficient API communication
- Data consistency and integrity
- Error handling for invalid inputs
- Scalable system architecture

Hardware and Software Requirements:
Software:
- Node.js
- React (Vite)
- MongoDB
- Express.js
- Code Editor (VS Code)

Hardware:
- Minimum 4GB RAM
- 1GB free disk space
- Modern web browser

Key Technologies and Design Aspects:
• **Frontend**: React with Tailwind CSS for responsive UI
• **Backend**: Node.js with Express for API handling
• **Database**: MongoDB using Mongoose ODM
• **API Communication**: Axios for sending HTTP requests
• **Routing**: React Router for navigation

• **Database Design**:
Uses collections for Products, Orders, Users, and Delivery Agents
Establishes relationships using ObjectIds

• **Object-Oriented Design:**
Components represent real-world entities (Product, Order, User)
Modular structure with reusable components

• **REST Architecture:**
Separate API endpoints for each operation
Clean separation of frontend and backend

Chapter 3 :PROGRAMMING

3.1 App.jsx

```
import { BrowserRouter, Routes, Route } from "react-router-dom";

import Login from "./pages/Login";

import SellerDashboard from "./pages/seller/SellerDashboard";
import AddProduct from "./pages/seller/AddProduct";
import ProductList from "./pages/seller/ProductList";

import AdminDashboard from "./pages/admin/AdminDashboard";
import AdminOrders from "./pages/admin/AdminOrders";
import DeliveryAgents from "./pages/admin/DeliveryAgents";

import BuyerDashboard from "./pages/buyer/BuyerDashboard";
import CreateOrder from "./pages/buyer/CreateOrder";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Login />} />

        <Route path="/seller" element={<SellerDashboard />} />
        <Route path="/seller/add-product" element={<AddProduct />} />
        <Route path="/seller/products" element={<ProductList />} />

        <Route path="/admin" element={<AdminDashboard />} />
        <Route path="/admin/orders" element={<AdminOrders />} />
        <Route path="/admin/delivery-agents" element={<DeliveryAgents />} />

        <Route path="/buyer" element={<BuyerDashboard />} />
        <Route path="/buyer/create-order" element={<CreateOrder />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

3.2 axios.js (API Configuration)

```
import axios from "axios";

const api = axios.create({
  baseURL: "http://localhost:5000/api"
});

export default api;
```

3.3 Login.jsx (Role-Based Login)

```
import { useState } from "react";
import { useNavigate } from "react-router-dom";

const Login = () => {
  const navigate = useNavigate();
  const [role, setRole] = useState("seller");

  const handleLogin = () => {
    if (role === "admin") navigate("/admin");
    else if (role === "seller") navigate("/seller");
    else navigate("/buyer");
  };

  return (
    <div>
      <h1>Login</h1>

      <select onChange={(e) => setRole(e.target.value)}>
        <option value="admin">Admin</option>
        <option value="seller">Seller</option>
        <option value="buyer">Buyer</option>
      </select>

      <button onClick={handleLogin}>Login</button>
    </div>
  );
};

export default Login;
```

3.4 SellerDashboard.jsx

```
import { useEffect, useState } from "react";
import api from "../../api/axios";

const SellerDashboard = () => {
  const [stats, setStats] = useState({
    totalProducts: 0,
    totalValue: 0
  });

  useEffect(() => {
    api.get("/seller/products").then((res) => {
      const products = Array.isArray(res.data) ? res.data : [];

      const totalProducts = products.length;
      const totalValue = products.reduce(
        (sum, p) => sum + (p.price || 0),
        0
      );

      setStats({ totalProducts, totalValue });
    });
  }, []);

  return (
    <div>
      <h2>Total Products: {stats.totalProducts}</h2>
      <h2>Total Value: ₹{stats.totalValue}</h2>
    </div>
  );
};

export default SellerDashboard;
```

3.5 AdminOrders.jsx (Core Logic)

```
const approveOrder = async (id) => {
  await api.put(`/admin/orders/${id}/approve`);
};

const assignAgent = async (orderId, agentId) => {
  await api.put(`/admin/orders/${orderId}/assign`, {
    deliveryAgentId: agentId
  });
};
```

```
const markDelivered = async (id) => {
  await api.put(`/admin/orders/${id}/delivered`);
};
```

3.6 Backend – Product Model (Mongoose)

```
import mongoose from "mongoose";

const productSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  price: {
    type: Number,
    required: true
  },
  category: {
    type: String
  },
  stock: {
    type: Number,
    default: 0
  },
  seller: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true
  }
}, { timestamps: true });

export default mongoose.model("Product", productSchema);
```

3.7 Backend – Order Model

```
import mongoose from "mongoose";

const orderSchema = new mongoose.Schema({
  product: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Product"
  },
  buyer: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User"
  },
```

```
  seller: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User"
  },
  deliveryAgent: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "DeliveryAgent"
  },
  status: {
    type: String,
    enum: [
      "Pending Approval",
      "Approved",
      "Out for Delivery",
      "Delivered"
    ],
    default: "Pending Approval"
  }
}, { timestamps: true });

export default mongoose.model("Order", orderSchema);
```

3.8 Backend – Seller Routes

```
import express from "express";
import Product from "../models/Product.js";

const router = express.Router();

// Get all products
router.get("/products", async (req, res) => {
  const products = await Product.find();
  res.json(products);
});

// Add product
router.post("/add-product", async (req, res) => {
  try {
    const product = new Product(req.body);
    await product.save();
    res.json(product);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

```
// Delete product
router.delete("/products/:id", async (req, res) => {
  await Product.findByIdAndDelete(req.params.id);
  res.json({ message: "Deleted" });
});

export default router;
```

3.9 Backend – Admin Routes

```
import express from "express";
import Order from "../models/Order.js";

const router = express.Router();

// Get all orders
router.get("/orders", async (req, res) => {
  const orders = await Order.find()
    .populate("product")
    .populate("buyer");
  res.json(orders);
});

// Approve order
router.put("/orders/:id/approve", async (req, res) => {
  await Order.findByIdAndUpdate(req.params.id, {
    status: "Approved"
  });
  res.json({ message: "Approved" });
});

// Assign delivery agent
router.put("/orders/:id/assign", async (req, res) => {
  await Order.findByIdAndUpdate(req.params.id, {
    deliveryAgent: req.body.deliveryAgentId,
    status: "Out for Delivery"
  });
  res.json({ message: "Assigned" });
});

// Mark delivered
router.put("/orders/:id/delivered", async (req, res) => {
  await Order.findByIdAndUpdate(req.params.id, {
    status: "Delivered"
  });
  res.json({ message: "Delivered" });
```

```
});

export default router;
```

3.10 Frontend – AddProduct.jsx

```
import { useState } from "react";
import api from "../../api/axios";

const AddProduct = () => {
  const [form, setForm] = useState({
    name: "",
    price: "",
    category: "",
    stock: ""
  });

  const handleSubmit = async () => {
    try {
      await api.post("/seller/add-product", form);
      alert("Product added");
    } catch {
      alert("Failed to add product");
    }
  };

  return (
    <div>
      <h2>Add Product</h2>

      <input
        placeholder="Name"
        onChange={(e) =>
          setForm({ ...form, name: e.target.value })
        }
      />

      <input
        placeholder="Price"
        onChange={(e) =>
          setForm({ ...form, price: e.target.value })
        }
      />

      <button onClick={handleSubmit}>Add</button>
    </div>
```

```
  );
};

export default AddProduct;
```

3.11 Frontend – ProductList.jsx

```
import { useEffect, useState } from "react";
import api from "../../api/axios";

const ProductList = () => {
  const [products, setProducts] = useState([]);

  useEffect(() => {
    api.get("/seller/products")
      .then(res => setProducts(res.data));
  }, []);

  return (
    <div>
      <h2>Products</h2>

      {products.map(p => (
        <div key={p._id}>
          <p>{p.name} - ₹{p.price}</p>
        </div>
      ))}
    </div>
  );
};

export default ProductList;
```

3.12 Frontend – Buyer CreateOrder.jsx

```
import { useState, useEffect } from "react";
import api from "../../api/axios";

const CreateOrder = () => {
  const [products, setProducts] = useState([]);
  const [selected, setSelected] = useState("");

  useEffect(() => {
    api.get("/seller/products")
      .then(res => setProducts(res.data));
  }, []);
```

```
  const createOrder = async () => {
    await api.post("/buyer/create-order", {
      productId: selected
    });
  };

  return (
    <div>
      <h2>Create Order</h2>

      <select onChange={(e) => setSelected(e.target.value)}>
        <option>Select Product</option>
        {products.map(p => (
          <option key={p._id} value={p._id}>
            {p.name}
          </option>
        ))}
      </select>

      <button onClick={createOrder}>Order</button>
    </div>
  );
};

export default CreateOrder;
```

3.13 Layout.jsx (Sidebar UI)

```
import { Link } from "react-router-dom";

const Layout = ({ children }) => {
  return (
    <div className="flex">

      <div className="w-64 bg-gray-200 p-4">
        <Link to="/admin">Dashboard</Link>
        <Link to="/admin/orders">Orders</Link>
      </div>

      <div className="flex-1 p-4">
        {children}
      </div>
    </div>
  );
};
```

```
export default Layout;
```

3.14 Server.js (Backend Entry Point)

```
import express from "express";
import mongoose from "mongoose";
import cors from "cors";

import sellerRoutes from "./routes/seller.js";
import adminRoutes from "./routes/admin.js";

const app = express();

app.use(cors());
app.use(express.json());

mongoose.connect("mongodb://localhost:27017/logistics");

app.use("/api/seller", sellerRoutes);
app.use("/api/admin", adminRoutes);

app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

Chapter 4 :RESULTS

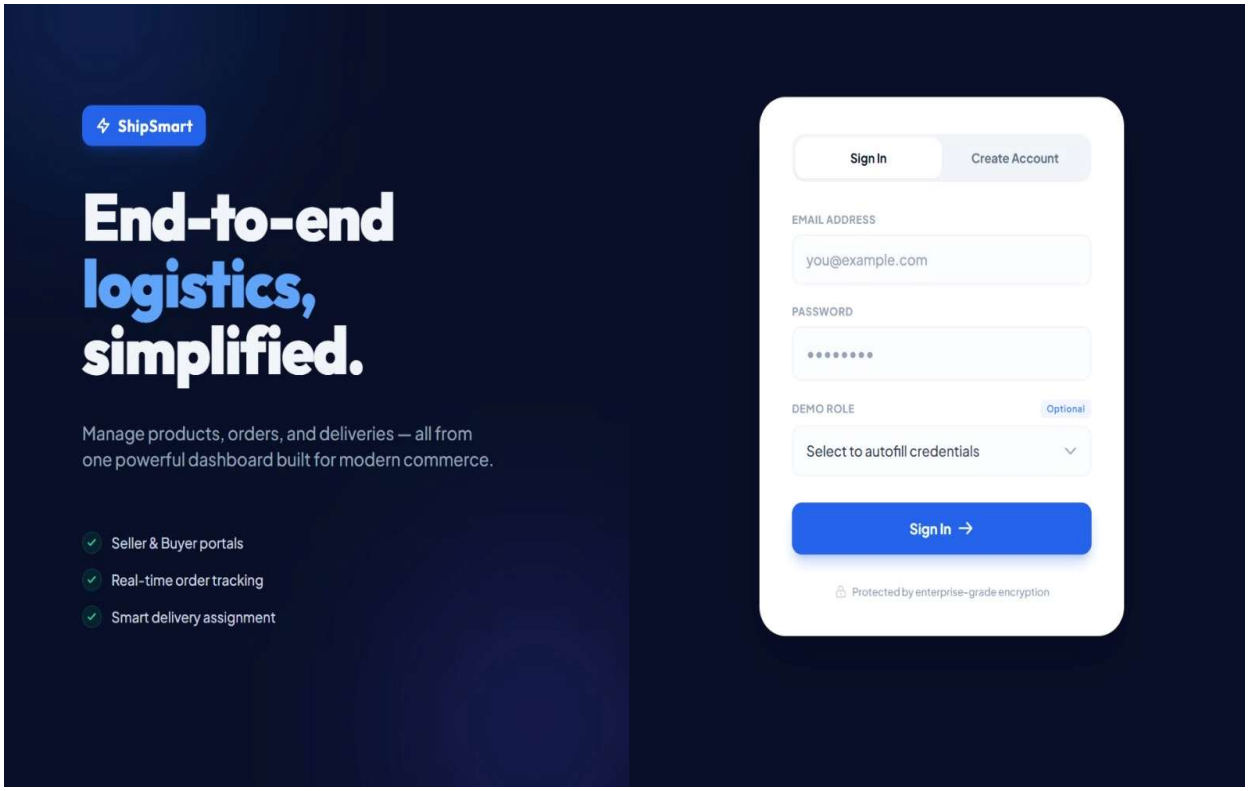


Fig. 1 Logging in

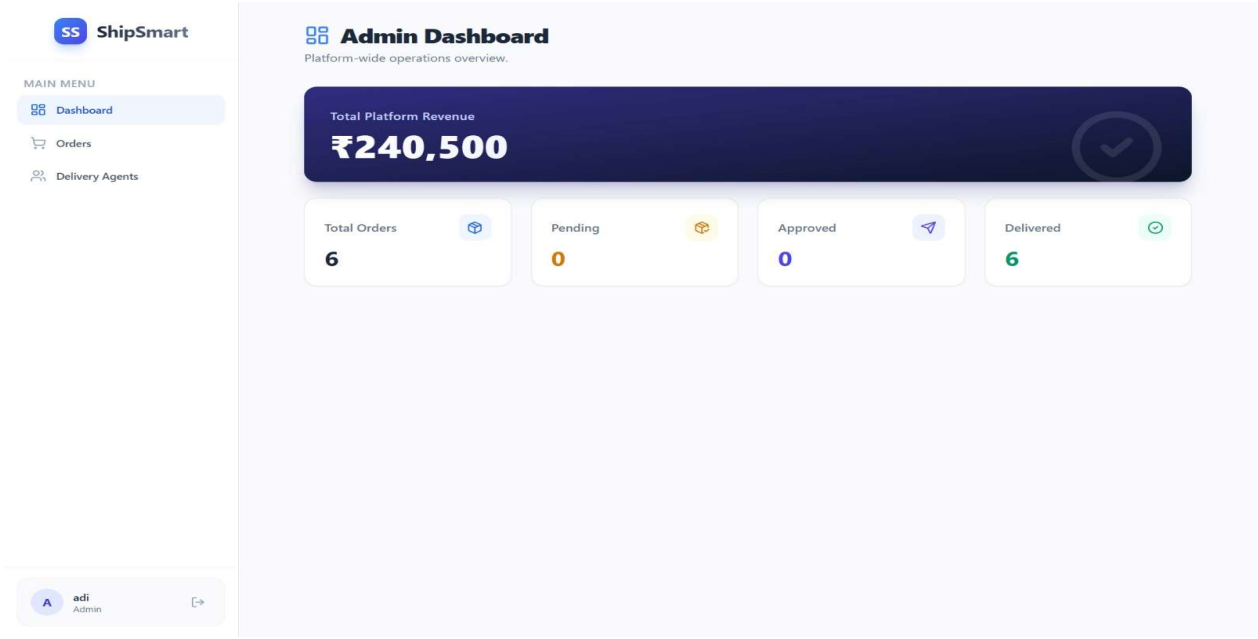


Fig. 2 Admin Dasboard

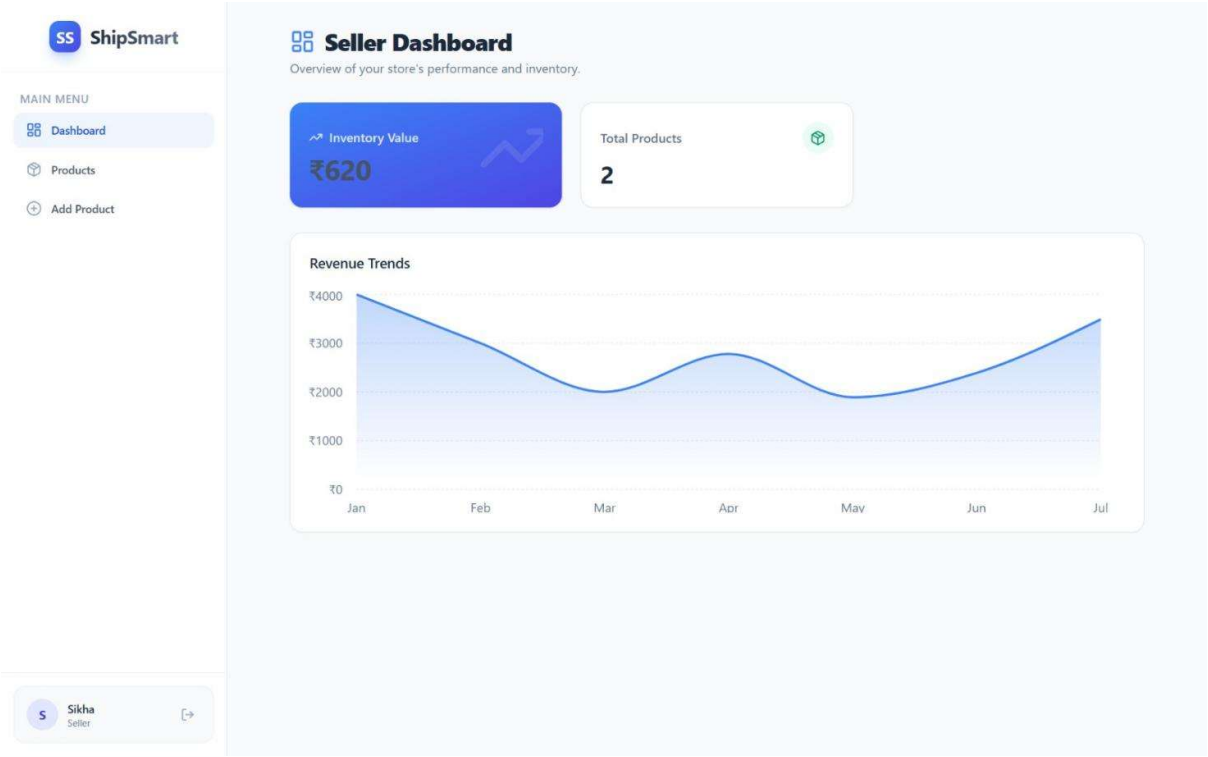
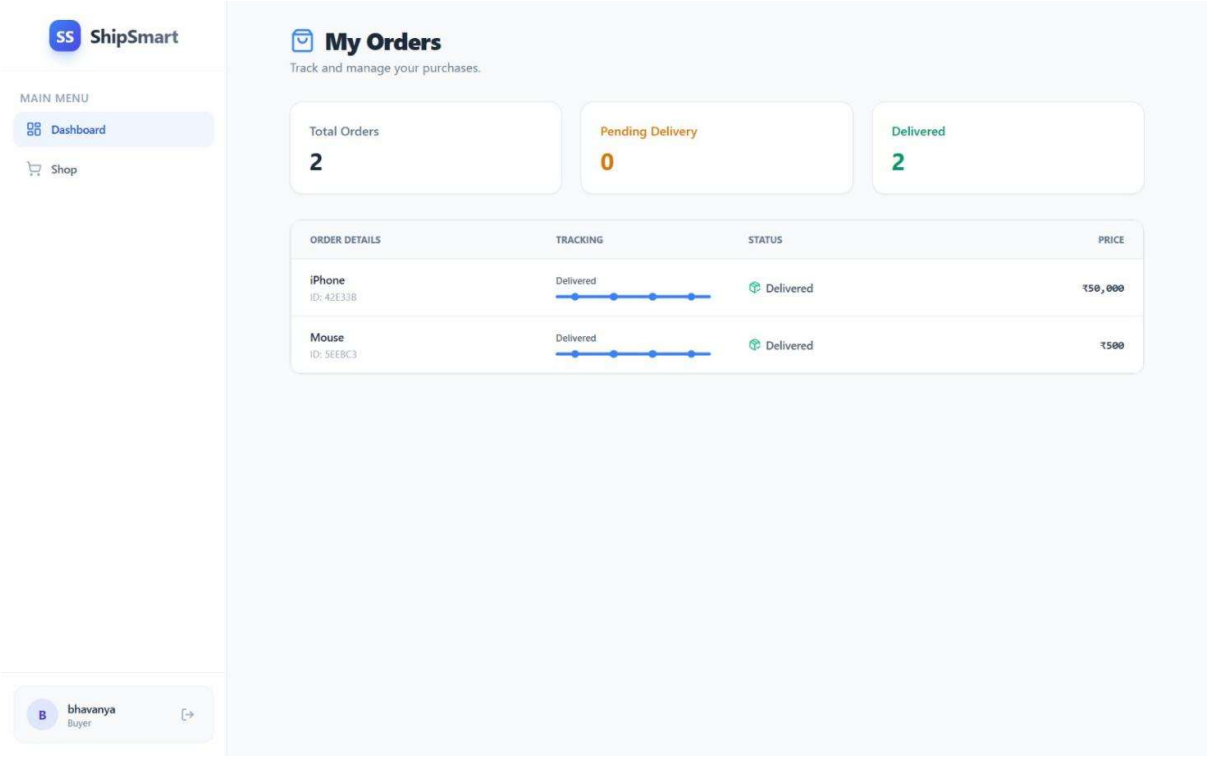


Fig. 3 Seller Dasboard



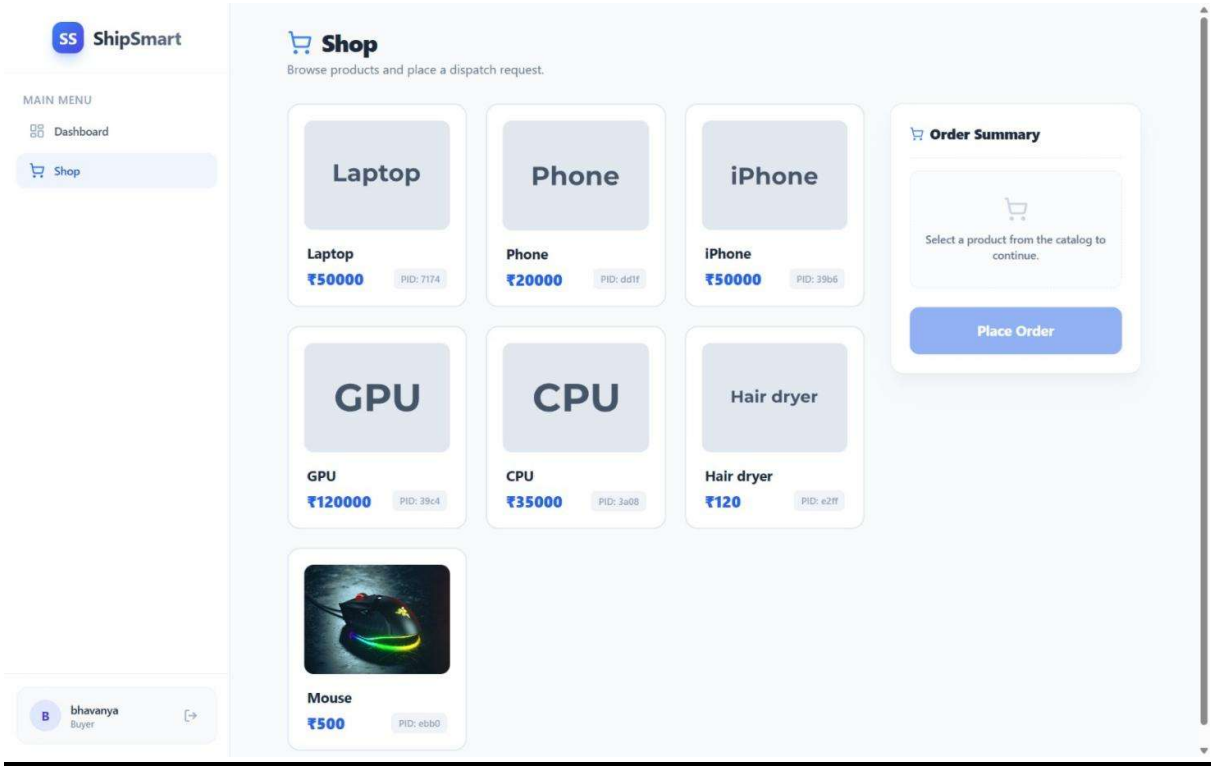


Fig. 4 Buyer Dasboard

Chapter 5 : CONCLUSION

The Logistics and E-Commerce Management System developed in this project successfully demonstrates the integration of modern web technologies with database management concepts to create a functional, user-friendly application. The system effectively manages the interaction between multiple user roles—Admin, Seller, and Buyer—while ensuring a smooth and structured workflow for product handling and order processing.

Through this project, essential operations such as product management, order creation, approval, delivery assignment, and status tracking have been implemented in a systematic manner. The use of MongoDB as a database ensures efficient storage and retrieval of data, while Node.js and Express handle backend logic and API communication. On the frontend, React provides a responsive and dynamic user interface, enhancing user experience.

One of the key achievements of this project is the implementation of a clear order lifecycle, which improves transparency and accountability. Additionally, the modular design of the system allows for easy scalability and future enhancements, such as implementing secure authentication (JWT), payment integration, and real-time tracking.

Overall, this project not only fulfills its objective of building a logistics management system but also provides practical exposure to full-stack development and database management concepts. It highlights the importance of structured design, efficient data handling, and seamless integration between frontend and backend systems, making it a valuable learning experience and a strong foundation for real-world applications.

References

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019).
   Database System Concepts (7th ed.). McGraw-Hill Education.

2. Elmasri, R., & Navathe, S. B. (2016).
   Fundamentals of Database Systems (7th ed.). Pearson.

3. Mozilla Developer Network (MDN).
   JavaScript Documentation.
   https://developer.mozilla.org/

4. React Documentation.
   React – A JavaScript library for building user interfaces.
   https://react.dev/

5. Node.js Documentation.
   Node.js Official Documentation.
   https://nodejs.org/

6. Express.js Documentation.
   Fast, unopinionated, minimalist web framework for Node.js.
   https://expressjs.com/

7. MongoDB Documentation.
   MongoDB Manual and Guides.
   https://www.mongodb.com/docs/

8. Axios Documentation.
   Promise-based HTTP Client for the Browser and Node.js.
   https://axios-http.com/

9. Tailwind CSS Documentation.
   Utility-first CSS framework.
   https://tailwindcss.com/

10. W3Schools.
    Web Development Tutorials.
    https://www.w3schools.com/